



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

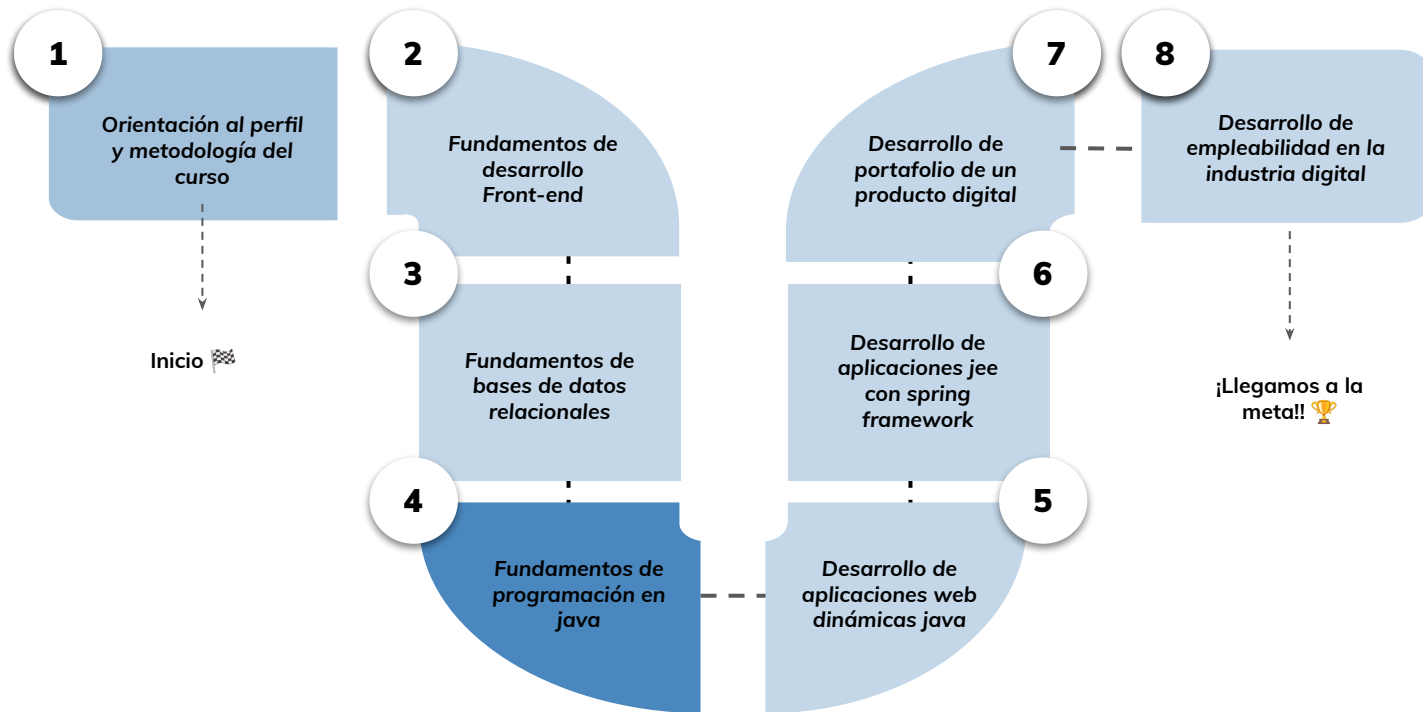


› El paradigma de orientación a objetos algoritmos- Parte 1

Aprendizaje Esperado 5: Utilizar elementos de la programación orientada a objetos para la implementación de una pieza de software que da solución a un problema de baja complejidad.

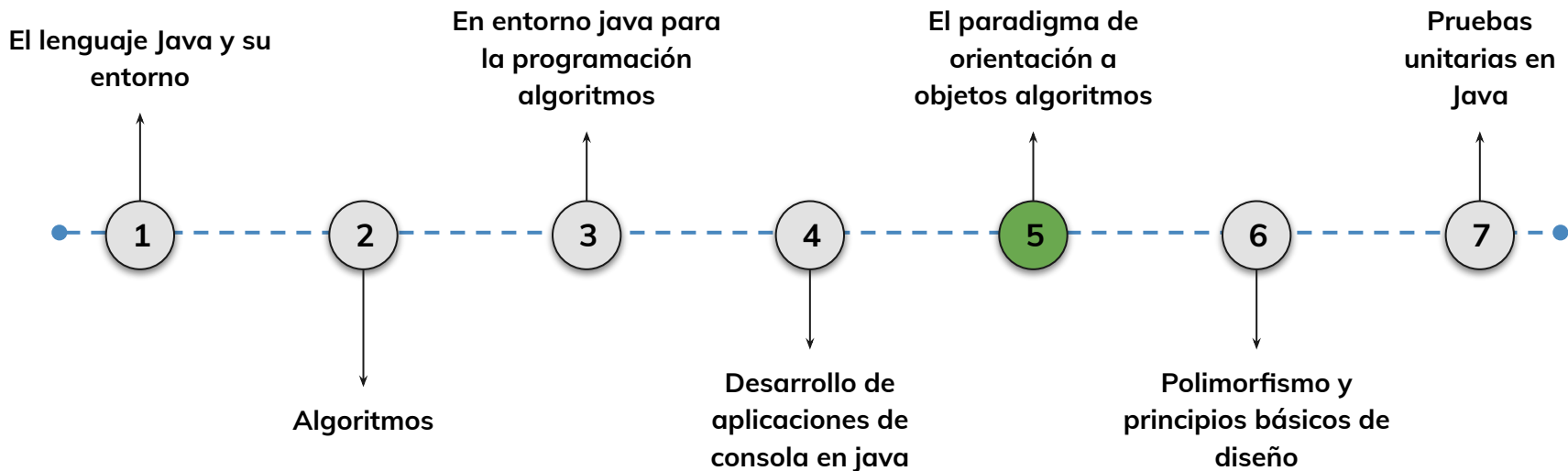
Hoja de ruta

¿Cuáles skills conforman el programa? **Desarrollo de Aplicaciones Full Stack Java Trainee**



Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

5.

Comprensión del paradigma de orientación a objetos y su aplicación práctica

Durante esta clase conoceremos los fundamentos de la programación orientada a objetos (POO) y su implementación en Java. Abordaremos los conceptos de clases, objetos, atributos, métodos y encapsulamiento, a través de ejemplos reales que nos permitirán modelar situaciones del mundo real en software.

El Paradigma de Orientación a Objetos

Definición de clases y creación de objetos

Atributos como estado de los objetos

Constructores y encapsulamiento

Métodos constructores y uso de this

Modificadores de acceso: public, private y protected

Objetivos de aprendizaje

¿Qué aprenderás?

- Reconocer la importancia de la Orientación a Objetos
- Comprender el concepto e implementación de las Clases y los Objetos
- Comprender e implementar los métodos constructores
- Comprender el concepto de encapsulamiento
- Aprender la utilización de accesadores y mutadores

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

- Aprendimos a documentar código con JavaDoc
- Usamos etiquetas clave como @param y @return
- Aplicamos estándares de formato y organización
- Depuramos un error con Eclipse Debug

< > Rompehielo

¿Y tú que piensas?: 🙌

Veán con atención la imagen y respondan en el chat y levantando la mano:

- ¿Son los mismos objetos?
- ¿Qué características tienen en común?
- ¿En qué se diferencian?
- ¿Cómo los clasificarías/agruparías?



Sedan



Wagon/Minivan/MPV



SUV



Hatchback



Cabriolet/Coupe



Sport/Supercar



Pickup



4WD



Mini Truck



Micro/Electric



Van



Campervan

➤ El Paradigma de Orientación a Objetos

< > El Paradigma de Orientación a Objetos

¿Qué es un Paradigma de programación?:

Es una manera o estilo de programación. Existen diferentes formas de diseñar un programa y varios modos de trabajar para obtener los resultados que necesitan los programadores.

La Programación Orientada a Objetos (**POO**) es un paradigma que modela elementos del mundo real como objetos que tienen propiedades y comportamientos. Esto permite representar entidades y sus características para crear programas más intuitivos.

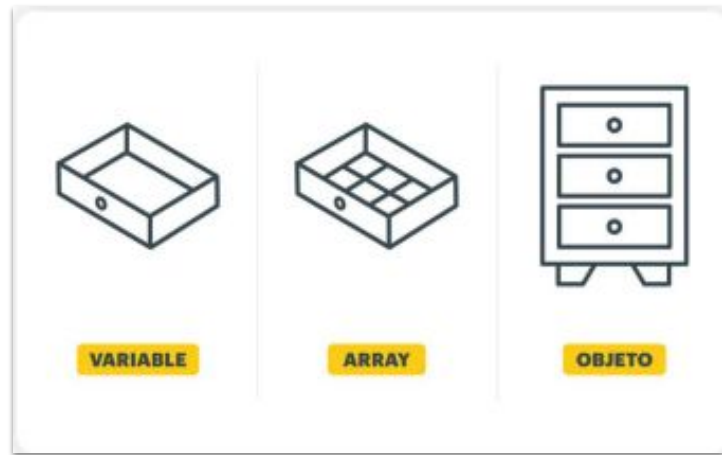
También permite organizar el código en módulos o **clases** reutilizables, y sus respectivas instancias y **objetos**, lo que facilita la creación y mantenimiento de programas complejos.



< > El Paradigma de Orientación a Objetos

La Programación Orientada a objetos permite que el código sea reutilizable, organizado y fácil de mantener.

Sigue el principio de desarrollo de software utilizado por muchos programadores **DRY (Don't Repeat Yourself)** , para evitar duplicar el código y crear de esta manera programas eficientes.



< > Clases

Son las plantillas o moldes que utilizaremos para crear objetos . Definen las propiedades y comportamientos que los objetos de esa clase tendrán. Entonces, una clase es una especie prototipo de objetos: define los **atributos** que componen ese tipo de objetos y los **métodos** que pueden emplearse para trabajar con esos objetos.

En su forma más simple, una clase se define por la palabra reservada **class** seguida del nombre de la clase. El **nombre de la clase debe empezar por mayúscula** . Si el nombre es compuesto, cada palabra debe empezar por mayúscula.

La definición de la clase se pone entre las llaves de apertura y cierre.

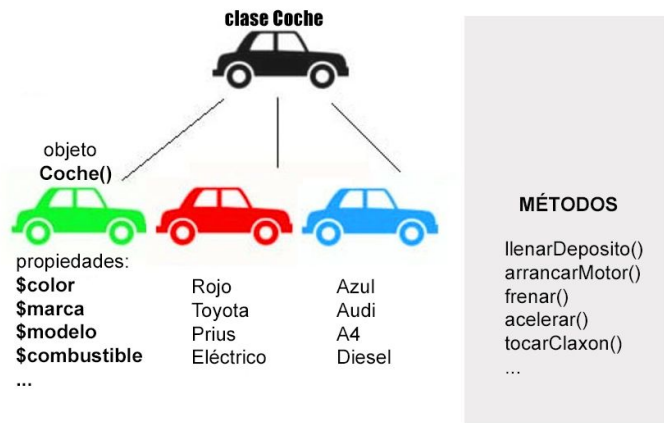
```
public class NombreClase {  
    // atributos ;  
    // constructores ;  
    // métodos propios ;  
}
```



< > Objetos

Son instancias específicas de una clase. Se crean a partir de la clase y tienen su propio estado y comportamiento.

- **Estado** : viene dado por sus **atributos** , que almacenan los datos del objeto. Por ejemplo, el color y marca de un coche.
- **Comportamiento** : viene dado por sus **métodos** que son las operaciones que puede realizar el objeto. Por ejemplo, el coche puede frenar y acelerar.



< > Estado de un objeto: Atributos

Los **atributos** son características comunes a todos los objetos.

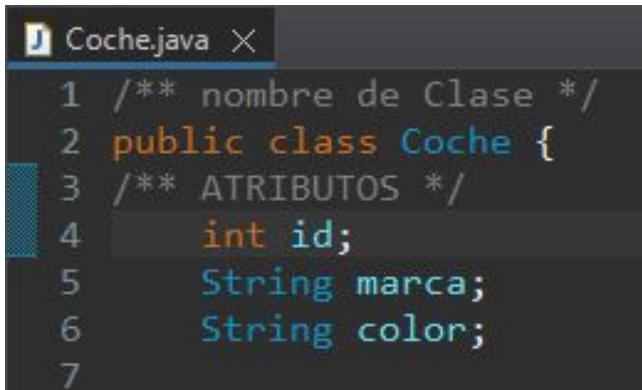
El **estado** o información de un objeto **se almacena en atributos**. Los atributos pueden ser de tipos primitivos de Java o del tipo de otros objetos.

La declaración de un atributo de un objeto tiene la siguiente forma:

<tipo de dato> <nombre del atributo> ;

<tipo> : indica la clase a la que pertenece el atributo definido.

<nombre> : puede ser cualquier identificador válido y denomina el atributo que está siendo declarado.



```
Coche.java X
1 /** nombre de Clase */
2 public class Coche {
3 /** ATRIBUTOS */
4     int id;
5     String marca;
6     String color;
7
```

LIVE CODING

Ejemplo en vivo

Billetera Virtual:

Vamos a crear una Wallet desde el comienzo! Para esto, crearemos una clase llamada Cuenta que manipulara los siguientes datos:

1. Atributos de la clase Cuenta:
 - Número de cuenta (int)
 - Titular (String)
 - Saldo (double)
2. Crear un objeto billetera1 y asignarle valores a estos atributos.

Tiempo: 25 minutos



Ejercicio N° 1

< > Cuenta Bancaria

Hora de codear: 🙌

La POO nos ayuda a programar pensando y modelando la realidad. Vamos a poner en práctica lo aprendido.

Consigna: ✍️

- Crea un proyecto llamado Banco.
- Crea una Clase llamada CuentaBancaria
- Agrega los siguientes atributos y dales un valor:
 - entero número de cuenta
 - doble saldo actual
 - cadena titular

Tiempo 🕒: 25 minutos



➤ Métodos de Clase: Constructores

< > Métodos de una Clase

¿Qué son?:

Son **funciones** o **procedimientos** que representan el **comportamiento** o las acciones que los objetos de esa clase pueden realizar.

Los métodos pueden aceptar parámetros como entrada, realizar cálculos, **modificar el estado de los objetos** y devolver o no resultados como salida.

Ayudan a organizar y modularizar el código, ya que permiten dividir la lógica del programa en tareas más pequeñas y manejables.

```
[<modificador> ] <nombre de método> ( <argumento> * ) {  
    <sentencia>;  
}
```



< > Métodos Constructores

¿Qué son?:

Los **constructores** son métodos **propios del objeto** que permiten la creación de dicho objeto. **A la creación de un objeto se le denomina instanciación.**

Los constructores se encargan de dar un **estado inicial** a nuestro objeto. Estos métodos tienen como característica principal **tener el mismo nombre que la propia clase**. Por lo tanto, debemos tener un método dentro de la clase “Coche” que se llame igual que ella.



< > Métodos Constructores

El método constructor **se ejecuta cada vez que se instancia un objeto** de la clase. Este método se utiliza para **inicializar** los atributos del objeto que se crea.

Para diferenciar entre los atributos del objeto y parámetros del método constructor, se utiliza la palabra **this**.

La instancia de un objeto consiste en **asignar un espacio de memoria** al que se hace referencia con el nombre del objeto.

```
8  /**
9   * constructor vacío o por defecto
10  */
11  public Coche () {
12
13  }
14  /**
15   * constructor parametrizado
16  */
17  public Coche(int id, String marca, String color) {
18      this.id = id;
19      this.marca = marca;
20      this.color = color;
21  }
22
23 }
24
```



< > Métodos Constructores

Para crear objetos, basta con declarar una variable de alguno de los tipos de las clases definidas:

NombreClase nombreObjeto = new nombreClase();

Para crear el objeto y asignar un espacio de memoria es necesario utilizar el operador **new**. El operador **new instancia el objeto** y reserva espacio en memoria para los atributos y devuelve una referencia que se guarda en la variable.

A partir de este momento los objetos ya pueden ser referenciados por su nombre (miAuto).

```
public class Main {  
    public static void main(String[] args) {  
        Coche miAuto = new Coche();  
    }  
}
```



< > Métodos Constructores

También podemos utilizar el método constructor parametrizado para instanciar un objeto, escribiendo los parámetros que le darán valor a cada atributo.

```
2 public class Main {  
3  
4     public static void main(String[] args) {  
5  
6         Coche miAuto = new Coche(123, "Nissan", "Negro");  
7     }  
8  
9 }  
10
```



LIVE CODING

Ejemplo en vivo

Billetera Virtual:

Ahora ya podemos construir un objeto Cuenta! Veamoslo practicando:

1. Agregar métodos constructores que inicialicen los atributos numeroCuenta, saldo y nombre(titular).
2. Instanciar una Cuenta.
3. Mostrar los valores de los atributos por pantalla.

Tiempo: 25 minutos



Ejercicio N° 1

< > CuentaBancaria 2.0

¡Vamos al Banco!: 🙌

Vamos a mejorar la clase CuentaBancaria creada en el ejercicio de la clase anterior.

Consigna: 📝

- Agregar métodos constructores
- Instanciar un objeto CuentaBancaria y darle valor a sus atributos.
- Mostrar los datos por pantalla.

Tiempo 🕒: 25 minutos



< > Rompehielo

¿Y tú que piensas?: 🙌

Veán con atención la imagen y respondan en el chat y levantando la mano:

- ¿Son los mismos objetos?
- ¿Qué características tienen en común?
- ¿En qué se diferencian?
- ¿Cómo los clasificarías/agruparías?



Sedan



Wagon/Minivan/MPV



SUV



Hatchback



Cabriolet/Coupe



Sport/Supercar



Pickup



4WD



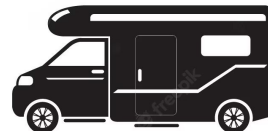
Mini Truck



Micro/Electric



Van



Campervan

< > El Paradigma de Orientación a Objetos

¿Qué es un Paradigma de programación?:

Es una manera o estilo de programación. Existen diferentes formas de diseñar un programa y varios modos de trabajar para obtener los resultados que necesitan los programadores.

La Programación Orientada a Objetos (**POO**) es un paradigma que modela elementos del mundo real como objetos que tienen propiedades y comportamientos. Esto permite representar entidades y sus características para crear programas más intuitivos.

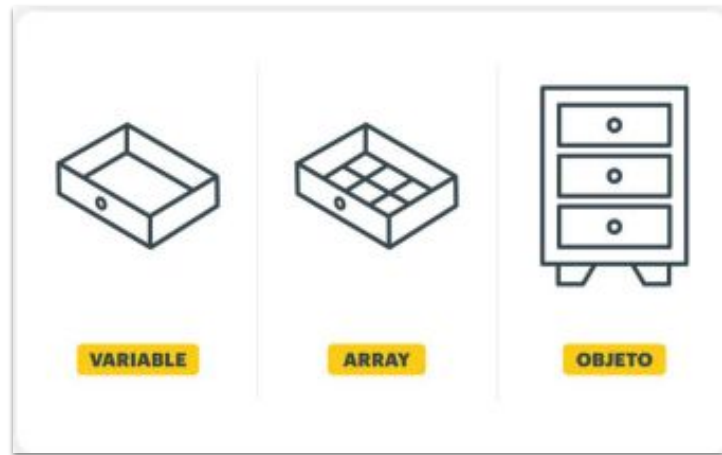
También permite organizar el código en módulos o **clases** reutilizables, y sus respectivas instancias y **objetos**, lo que facilita la creación y mantenimiento de programas complejos.



< > El Paradigma de Orientación a Objetos

La Programación Orientada a objetos permite que el código sea reutilizable, organizado y fácil de mantener.

Sigue el principio de desarrollo de software utilizado por muchos programadores **DRY (Don't Repeat Yourself)** , para evitar duplicar el código y crear de esta manera programas eficientes.



< > Clases

Son las plantillas o moldes que utilizaremos para crear objetos . Definen las propiedades y comportamientos que los objetos de esa clase tendrán. Entonces, una clase es una especie prototipo de objetos: define los **atributos** que componen ese tipo de objetos y los **métodos** que pueden emplearse para trabajar con esos objetos.

En su forma más simple, una clase se define por la palabra reservada **class** seguida del nombre de la clase. El **nombre de la clase debe empezar por mayúscula** . Si el nombre es compuesto, cada palabra debe empezar por mayúscula.

La definición de la clase se pone entre las llaves de apertura y cierre.

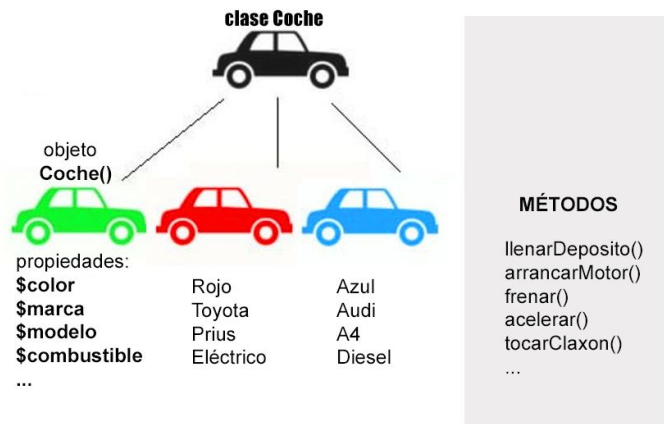
```
public class NombreClase {  
    // atributos ;  
    // constructores ;  
    // métodos propios ;  
}
```



< > Objetos

Son instancias específicas de una clase. Se crean a partir de la clase y tienen su propio estado y comportamiento.

- **Estado** : viene dado por sus **atributos** , que almacenan los datos del objeto. Por ejemplo, el color y marca de un coche.
- **Comportamiento** : viene dado por sus **métodos** que son las operaciones que puede realizar el objeto. Por ejemplo, el coche puede frenar y acelerar.



< > Estado de un objeto: Atributos

Los **atributos** son características comunes a todos los objetos.

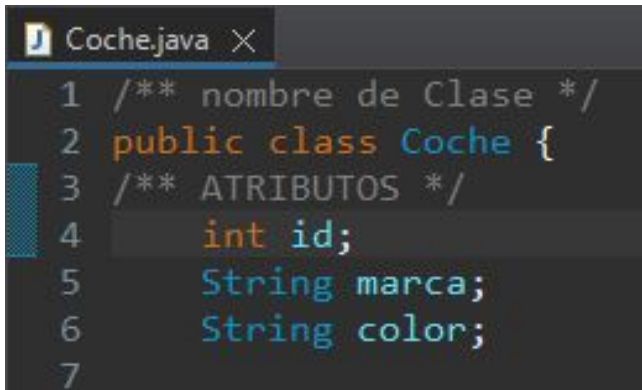
El **estado** o información de un objeto **se almacena en atributos**. Los atributos pueden ser de tipos primitivos de Java o del tipo de otros objetos.

La declaración de un atributo de un objeto tiene la siguiente forma:

<tipo de dato> <nombre del atributo> ;

<tipo> : indica la clase a la que pertenece el atributo definido.

<nombre> : puede ser cualquier identificador válido y denomina el atributo que está siendo declarado.



```
Coche.java X
1 /** nombre de Clase */
2 public class Coche {
3 /** ATRIBUTOS */
4     int id;
5     String marca;
6     String color;
7 }
```

< > Cuenta Bancaria

Hora de codear: 🙌

La POO nos ayuda a programar pensando y modelando la realidad. Vamos a poner en práctica lo aprendido.

Consigna: ✍️

- Crea un proyecto llamado Banco.
- Crea una Clase llamada CuentaBancaria
- Agrega los siguientes atributos y dales un valor:
 - entero número de cuenta
 - doble saldo actual
 - cadena titular

Tiempo 🕒: 25 minutos



< > Métodos de una Clase

¿Qué son?:

Son **funciones** o **procedimientos** que representan el **comportamiento** o las acciones que los objetos de esa clase pueden realizar.

Los métodos pueden aceptar parámetros como entrada, realizar cálculos, **modificar el estado de los objetos** y devolver o no resultados como salida.

Ayudan a organizar y modularizar el código, ya que permiten dividir la lógica del programa en tareas más pequeñas y manejables.

```
[<modificador> ] <nombre de método> ( <argumento> * ) {  
    <sentencia>;  
}
```



< > Métodos Constructores

¿Qué son?:

Los **constructores** son métodos **propios del objeto** que permiten la creación de dicho objeto. **A la creación de un objeto se le denomina instanciación.**

Los constructores se encargan de dar un **estado inicial** a nuestro objeto. Estos métodos tienen como característica principal **tener el mismo nombre que la propia clase**. Por lo tanto, debemos tener un método dentro de la clase “Coche” que se llame igual que ella.



< > Métodos Constructores

El método constructor **se ejecuta cada vez que se instancia un objeto** de la clase. Este método se utiliza para **inicializar** los atributos del objeto que se crea.

Para diferenciar entre los atributos del objeto y parámetros del método constructor, se utiliza la palabra **this**.

La instancia de un objeto consiste en **asignar un espacio de memoria** al que se hace referencia con el nombre del objeto.

```
8  /**
9   * constructor vacío o por defecto
10  */
11  public Coche () {
12
13  }
14  /**
15   * constructor parametrizado
16  */
17  public Coche(int id, String marca, String color) {
18      this.id = id;
19      this.marca = marca;
20      this.color = color;
21  }
22
23 }
24
```



< > Métodos Constructores

Para crear objetos, basta con declarar una variable de alguno de los tipos de las clases definidas:

NombreClase nombreObjeto = new nombreClase();

Para crear el objeto y asignar un espacio de memoria es necesario utilizar el operador **new**. El operador **new instancia el objeto** y reserva espacio en memoria para los atributos y devuelve una referencia que se guarda en la variable.

A partir de este momento los objetos ya pueden ser referenciados por su nombre (miAuto).

```
public class Main {  
    public static void main(String[] args) {  
        Coche miAuto = new Coche();  
    }  
}
```



< > Métodos Constructores

También podemos utilizar el método constructor parametrizado para instanciar un objeto, escribiendo los parámetros que le darán valor a cada atributo.

```
2 public class Main {  
3  
4     public static void main(String[] args) {  
5  
6         Coche miAuto = new Coche(123, "Nissan", "Negro");  
7     }  
8  
9 }  
10
```



< > CuentaBancaria 2.0

¡Vamos al Banco!: 🙌

Vamos a mejorar la clase CuentaBancaria creada en el ejercicio de la clase anterior.

Consigna: 📝

- Agregar métodos constructores
- Instanciar un objeto CuentaBancaria y darle valor a sus atributos.
- Mostrar los datos por pantalla.

Tiempo 🕒: 25 minutos



➤ Encapsulamiento

< > Encapsulamiento

¿Qué es?

El encapsulamiento **oculta** lo que hace un objeto de lo que hacen otros objetos y del mundo exterior, por lo que se denomina también **ocultación de datos** .

El encapsulamiento busca de alguna forma **controlar el acceso a los datos** que **conforman un objeto** o instancia, de este modo podríamos decir que una clase y por ende sus objetos que hacen uso de **modificadores de acceso** (especialmente privados) **son objetos encapsulados** .

Ocultar el estado interno y hacer que toda interacción sea a través de los métodos del objeto es un mecanismo conocido como **encapsulamiento de datos** .



< > **Modificadores de acceso**

¿Para qué se utilizan?

Permiten dar un **nivel de seguridad mayor** a nuestras aplicaciones **restringiendo el acceso** a diferentes atributos, métodos, constructores asegurándonos que el usuario deba seguir una "ruta" especificada por nosotros para acceder a la información.

Todas las clases poseen diferentes niveles de acceso en función del modificador de acceso (**visibilidad**):

- **public**
- **private**
- **protected**



< > Modificadores de acceso

Modificador public:

El modificador de acceso public es **el más permisivo** de todos, esto quiere decir que si un componente de una clase es public, tendremos **acceso a él desde cualquier clase** o instancia sin importar el paquete o procedencia de ésta.

```
1 /** nombre de Clase */
2 public class Coche {
3     /** ATRIBUTOS */
4     public int id;
5     public String marca;
6     public String color;
7 }
```



< > Modificadores de acceso

Modificador private:

En Java **es el más restrictivo de todos** , básicamente cualquier elemento de una clase que sea privado puede ser accedido **únicamente por la misma clase** . Es decir, si por ejemplo, si un atributo es privado solo puede ser accedido por los métodos o constructores que se encuentren en la misma clase. Ninguna otra clase, sin importar la relación que tengan, podrá tener acceso a ellos.

```
1 /** nombre de Clase */
2 public class Coche {
3     /** ATRIBUTOS */
4     private int id;
5     private String marca;
6     private String color;
7 }
```



< > Modificadores de acceso

Modificador protected:

El modificador de acceso protected nos **permite acceso** a los componentes con dicho modificador desde la misma clase, clases **del mismo paquete y clases que hereden de ella** (incluso en diferentes paquetes).

```
1 /** nombre de Clase */
2 public class Coche {
3     /** ATRIBUTOS */
4     protected int id;
5     protected String marca;
6     protected String color;
7 }
```



< > **Modificadores de acceso**

No son únicamente para los atributos, sino también para los métodos y procedimientos específicos de cada clase. El comportamiento de los componentes será el mismo que explicamos anteriormente.

Nota: Siempre se recomienda que los atributos de una clase sean **privados** y por tanto cada atributo debe tener sus propios métodos de acceso y consulta (los veremos en la próxima clase).

Nota 2: Siempre que se use una clase de otro paquete, se debe importar usando **import**. Cuando dos clases se encuentran en el mismo paquete no es necesario hacer el import pero esto no significa que se pueda acceder a sus componentes directamente.



LIVE CODING

Ejemplo en vivo

Billetera Virtual:

En esta breve ejemplo, vamos a colocar los modificadores de acceso necesarios a cada atributo y método de la clase Cuenta, y vamos a ver cómo se comporta.

1. Agregar modificador `public` y ejecutar el programa
2. Agregar modificador `private` y ejecutar el programa
3. Agregar modificador `protected` y ejecutar el programa

Tiempo:



Ejercicio N°

< > CuentaBancaria 3.0

Manos a la obra: 🙌

Ahora vamos a poner en práctica lo aprendido modificando la visibilidad de los atributos y métodos de la clase CuentaBancaria con la que venimos trabajando.

Consigna: 📝

- Colocar modificador private a los atributos
- Colocar modificador public a los métodos
- Crear un método público 'ingresarDinero' que permita ingresar un monto de dinero y sumarlo al atributo saldoActual.
- Imprimir los valores por pantalla

Tiempo 🕒: 30 minutos





Momento:

Time-out!



5 -10 min.



¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ **Comprendimos la importancia de la POO**
- ✓ **Conocimos la implementación de Clases y el estado y comportamiento de los Objetos**
- ✓ **Comprendimos el concepto e implementación de los métodos constructores**
- ✓ **Reconocimos la utilidad e implementación de los modificadores de acceso**



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

